

P1687R0

Summary of the Tooling Study Group's Pre-Cologne Telecons on Modules Tooling Interactions

Published Proposal, 2019-06-16

Authors:

[Bryce Adelstein Lelbach](#) (NVIDIA)

[Ben Craig](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

EWG, SG15

Table of Contents

1 Introduction

2 Summary of Meetings

3 Meeting Minutes

3.1 2019-03-08 Meeting Minutes

3.2 2019-03-22 Meeting Minutes

3.3 2019-04-05 Meeting Minutes

3.4 2019-04-12 Meeting Minutes

3.5 2019-04-26 Meeting Minutes

- 3.6 2019-05-10 Meeting Minutes
- 3.7 2019-05-24 Meeting Minutes
- 3.8 2019-06-07 Meeting Minutes

§ 1. Introduction

At the ISO C++ Kona 2019 meeting, the ISO C++ Committee’s Tooling Study Group, SG15, met to discuss concerns raised by various stakeholders about how modules would impact and interact with the broader C++ ecosystem (build systems, tools, other languages, etc). During that discussion, SG15 reached a consensus that the best way to prepare the C++ community for modules and ensure a smooth transition to modules over the next decade would be to prepare a C++ Ecosystem Technical Report on modules.

Since that meeting, the Tooling Study Group has held a series of telecons to plan, discuss, and brainstorm ideas for the proposed Technical Report. One of the outcomes of these telecons is [P1688](#), an outline and high-level design for the proposed Technical Report.

This document contains a summary and detailed minutes for each of the telecons.

§ 2. Summary of Meetings

Date	Agenda	Organizers
2019-03-08	— Schedule for pre-Cologne Tooling Study Group telecons. — Scope, priorities, and goals for the proposed C++ Ecosystem Technical Report.	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Ben Craig.
2019-03-22	— Suggested outline for the proposed C++ Ecosystem Technical.	— <i>Chairing</i>: Ben Craig. — <i>Minutes</i>: Tom Honermann.
2019-04-05	— P1689: Dependency Format Specification (Ben Boeckel). — Logistics of drafting the proposed C++ Ecosystem Technical Report	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Ben Craig.

	(format, repos, etc).	
2019-04-12	— GCC Module Mapping — P1184: A Module Mapper (Nathan Sidwell). — P1602: Make Me A Module (Nathan Sidwell). — New sg15@lists.isocpp.org mailing list.	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Ben Craig.
2019-04-26	— Distributed Build Systems (Ben Craig, Mathias Stearn). — Clang Modules and Module Maps (Richard Smith). — New sg15@lists.isocpp.org mailing list.	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Bryce Adelstein Lelbach, Ben Craig.
2019-05-10	— P1634: Module Naming (Corentin Jabot).	— <i>Chairing</i>: Ben Craig. — <i>Minutes</i>: Ben Craig.
2019-05-24	— Compiled Module Distribution (Olga Arkhipova).	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Ben Craig.
2019-06-07	— Plans for Pre-Cologne mailing papers. — Compiled Module Configuration (Michael Spencer). — New sg15@lists.isocpp.org mailing list.	— <i>Chairing</i>: Bryce Adelstein Lelbach. — <i>Minutes</i>: Ben Craig.

§ 3. Meeting Minutes

§ 3.1. 2019-03-08 Meeting Minutes

Attendance:

- Ben Craig.
- Bryce Adelstein Lelbach (NVIDIA).
- Isabella Muerte.
- Olga Arhipova (Microsoft).
- Ben Boeckel (Kitware).
- Boris Kolpackov (build2).
- Rene Rivera (Boost.Build).
- Richard Smith (Google).
- Tom Honermann (Synopsys).
- Mark Zeren (VMWare).
- Christof Meerwalk.
- Bruno Lopes (Apple).
- JF Bastien (Apple).
- Michael Spencer (Apple).
- Mathias Stearn (MongoDB).
- Corentin Jabot.
- Peter Bindels (TomTom).
- Steve Downey (Bloomberg).

Chair Notes (Bryce Adelstein Lelbach):

- For the purpose of our discussions, let's assume that a ISO Technical Report is the right type of document. If we find that this type of document doesn't meet what we need, we'll use another type of document.
- When should we aim to have the Technical Report completed?
- When C++20 is published?
- After C++20 is published?

- When it's ready!
- Should the Technical Report be a living document that is evolved/revised?
- Have it be versioned, not live-at-head?
- Who will use modules?
- Compiler vendors.
- Build system vendors.
- Tool vendors.
- Library vendors.
- Distribution vendors (ex: RPM, Debian, homebrew, vcpkg).
- End users.
- Other languages that want to interact with C++ modules.
- What concrete questions do we need to address in the Technical Report?
- Ex: Is shipping prebuilt BMIs best practice?
- How do we express module mappings to compilers?
- Specification of available modules from an installed package?
- What do we need to do to enable adoption of modules by existing build systems and tools?
- How can we best exploit them long term?
- How do I express that a header is modular (e.g. that you can do `import <foo>;` and `#include <foo>` can be treated as an import)?
- How do we avoid conflicts between module names?
- How do we maintain ABI compatibility with modules?
- How do we deal with conflicts between the compiler options that different producers of modules use and that consumers of modules use?
- What happens to the textual inclusion model in the modular world?
- When and how should you migrate to modules? When should you use `#include` , `import` , `import foo`? How do we communicate this to users?
- Should producers ship both headers and modules? When do you get rid of your headers?
- How do you make your code modular? How do you break cyclic includes? (Is this all in scope)
- How do people write headers that need to work for C++03/11/14/17 and be modular for C++20?

- What is a good size for a module? When is a module too big/small? How do modules scale (e.g. if I have a huge module and use just a few things, do I pay a price)? How do large modules affect build parallelism/dependency graphs?
- Don't be too prescriptive; make sure to provide alternatives and suggestions, not specific recipes.
- What specific use cases should the Technical Report address?
- Ex: Autocompletion in IDEs/editors.
- Hello world with modules.
- Ideal build setup look like for new projects in a modular world.
- Dependency scanning vs explicit module dependencies build example.
- Internal (modules that are part of my project) vs external (modules from outside of my project).
- Providing #includes for backwards compatibility
- Existing build systems consuming modules:
 - CMake.
 - Make.
 - Boost Build.
 - Internal company build systems.
 - autoconf.
 - Meson.
 - Ninja.
 - Scons.
 - Shell scripts.
 - Waf.
 - Bazel.
 - Buck.
 - Cargo.
 - Gulp.
 - Webpack.
 - Ant.
 - llbuild.
 - Evoke.

- qmake.
- MSBuild.
- Mixed build systems
 - Ex: CMake + qmake in the same
 - CMake + autoconf
 - Make + CMake
- Distributed builds (high bandwidth + high latency and low bandwidth + low latency):
 - icecc.
 - ccache.
 - sccache.
 - incredibuild.
 - distcc.
 - fastbuild.
 - Bazel remote build execution.
 - Internal company distributed build systems.
- Incremental builds
- Building module interfaces for tool purposes (code completion, etc).
- Static analysis tools:
 - Coverity.
 - Clang static analysis.
 - Grammatech (Aaron Ballman).
 - cppcheck.
- Other tools that traditional operate on a single TU:
 - SWIG.
 - Things based on clang tooling.
 - CastXML.
 - Qt moc.
 - Test mocking frameworks.

- Google mock.
- Things that generate code
 - protobuf.
- Test case reduction tools:
 - creduce.
 - delta.
- Modularizing libraries:
 - Header only libraries.
 - Boost.
- C++ language extensions:
 - CUDA (not mentioned by Bryce!).
 - Vulkan.
 - OpenCL.
 - SYCL.
- SG15 mailing list issues.
- Alternative mailing lists:
 - discourse.
 - Google Groups.
- Issues today with mailing list:
 - Emails bouncing to people, causing them to get unsubscribed.
 - Emails not getting delivered.
 - Dmarc emails.
- Action Item for Bryce:
 - Call Herb today about the mailing list.

Minutes (Ben Craig):

- Bryce: When should we have a TR go out?

- Mathias: Prefer to release it at the same time as the IS so that we have usage recommendation at the same time as we have the IS support
- Tom: Agree with Mathias, but not too concerned about the time. Let's work on it now and see when it is ready.
- Rene: Just saying that the argument for delaying the TR would also apply to not having modules in 20.
- Mathias: Yes, but since modules are currently in for 20, they should be useable when released. I think the TR is a major component of the usability story
- Olga: Hoping the doc evolves beyond C++20. It would be ideal for us to know everything, and we will get something useful, but hopefully we can evolve it past that.
- Bryce: open question: can a TR be a living document? Can it be evolved?
- Mathias: I think we can do versions, but probably not a live-at-head document.
- Rene: I just want to say that doing releases of it are best.
- Bryce: Let's assume that an ISO TR is the right kind of document.
- Corentin: There is no way we can do everything we want by the time C++20 is released. We should aim for a first release that is minimal. Maybe aim for more complete by C++21 or so. The document needs to give a set of good practices and bad practices. If module names don't match files, we can't change that, so we need to discourage that practice.
- Tom: Maybe today we can establish a priority list of things to address in the TR.
- Bryce: Just because the standard is out at C++20 doesn't mean that implementations will be available in C++20
- Steve D: The full standard may not be out in 2020, but people will start to want to work with them now in /std:c++2a modes.
- Mathias: All the big compilers have early implementations.
- Bryce: Work on defining scope, priorities, and goals for the Technical Report. Between now and next meeting, we will collate that list of things and try to prioritize those things.
- Ben Craig: Stakeholders that should be considered are compiler vendors, build tool vendors, library vendors end users
- Bryce: Should consider, and determine if package vendors are in scope (like system package managers).
- Ben Boeckel: Need to determine where package managers should put module interface units. That may be the extent of their involvement.
- Rene: If you make it clear what modules are not meant to cover, then we can scope better.

- Tom: Don't forget SWIG, static analyzers.
- Bryce: Agreed. Splitting build system vendors and tool vendors.
- Corentin: Distribution maintainers seems more accurate.
- Ben: depends on what you call anaconda, nix, and ports.
- Mathias: Maybe foreign function interfaces? May be out of scope?
- Bryce: Unsure if it falls into one of these categories.
- JF: For Clang modules, we've been bridging between objective C and C++.
- Olga: MSVC also considers modules as a way to interact with other languages.
- Mathias: Other languages may also produce modules to be consumed by C++.
- Corentin: I think I'd call them distribution too to avoid confusion with package.
- Steve D: filesystem hierarchy maintainers is more the case than distro maintainers. At least for first cut.
- Ben B: yes, thats linux-centric, but anything which deals with install prefixes cares.
- Steve D: BSD has a similar set of rules. Posix-ish.
- Ben B: theyll get ports since bsd is C for the long term.
- Bryce: What list of concrete open questions should we address? Example: Is shipping prebuild BMIs best practice?
- Mathias Stearn: Should these be a list of use-cases we want to address rather than questions?
- Boris Kolpackov: Here are some ideas on describing modular libraries in .pc (pkg-config) files: <https://build2.org/build2/doc/build2-build-system-manual.xhtml#cxx-modules-install>
- Ben B: My first thoughts on cmake's iface: <https://gitlab.kitware.com/ben.boeckel/cxx-modules-sandbox/blob/master/header-units/external/CMakeLists.txt>
- Ben B: Build tools that know about modules. Old build tools that don't. How do we tell compilers about modules (i.e. module maps). How to consume a library via modules (i.e. pkgconfig). pkgconfig tells me -I and -L flags. How do I convey that information? Specification of available modules from an install.
- Tom: What do we need to do to enable adoption? How can we adhere to what people do today?
- Steve D: How do I say something is eligible to be a header unit? How do I say where the interface for a module lives? When can a #include be converted to an import ?
- Corentin: How do we avoid conflicts between module names? How do we maintain ABI with modules?
- Bryce: Module ABI has multiple models of ownership. We'll need to discuss it at some point.

- Mathias: How close can we get to having modules be "self-descriptive" rather than relying on pkg-conf.
- Ben B: Would generating the pkg-conf from the source be sufficiently self-descriptive for you?
- Mathias: I meant as a distribution format. If module interface sources are distributed as archive files (still not agreed on, but the more I think about it, the more I like it), we can easily include as much metadata in any format we want. At that point, I don't think there is an advantage to shoehorning the metadata into pkg-conf is useful.
- Ben B: You might still need to know where to look for those files and their module name (if, e.g., you have a `__declspec(dllexport)` module name on Windows).
- Michael: The distribution format for module interfaces may as well be text.
- Mathias: Sure, but the same is true of the .pc files. My issue with .pc is that I don't think actual compile flags are the best way to convey metadata.
- Ben B: It wouldn't contain compile flags, but the same info that CMake stores in its usage requirements. Making the actual flags and rules is up to the consumer of the .pc file. Boris and I have very similar ideas here based on what I see in his link.
- Boris: Yeah, looks pretty similar.
- Mathias: Michael, in that situation, would you have each module unit distributed in separate files, even for libs with 100's of internal module partitions?
- Michael: Yeah.
- Mathias: (I know that is what we do today, so it wouldn't be much worse, this just seems like a reasonable thing to improve upon)
- Ben B: A tool to take a module file and 100 partitions and output a single miu would be useful. But can be done in the future. I'd like to see such a tool first before we try specifying it.
- Ben C: How can we best exploit modules long term?
- Olga: Maybe group these into questions per stakeholder?
- Tom: Multiple models for consuming modules. Separate compilation vs. textual inclusion. With separate compilation, you can get conflicting options. What kind of guidance do we give to avoid those conflicts so that the textual inclusion model can work. How do we deal with conflicts between the options that producers and consumers use and the impact to the ability to support a textual inclusion model?
- Mathias: Not sure if it is technically possible to support textual inclusion
- Tom: Good discussion to have at some other time.

- Bruno: How do we teach people to migrate to modules? When do we suggest to use old `#include`, vs header units, vs modules.
- Steve D: What are the techniques for fixing your code so that it can be modular? May be out of scope. Cycle breaking for example.
- Tom: How do people write headers that work for both C++20 and C++17? While still being modular?
- Corentin: How big should modules be? What makes them too big, too small?
- Bryce: Modules are scalable in clang and that big modules are fine?
- Richard: They are scalable, and they might be more efficient.
- Bryce: Is that expected to be true of all implementations?
- Richard: Unclear if it will be true for all implementations.
- Rene: We should be sure to stick to tooling as much as possible and not to get too much into software design.
- Mathias: Rene perhaps observations might work as notes in the TR?
- Rene: Sure. Note this is only for the software design POV. Prescribing for tooling is fine.
- Michael: It wouldn't be crazy to have a single text file represent multiple modules, it's just more non standard extensions to do it.
- Ben B: Yeah, but we can include "here's timings for project X using {tiny,large} modules for {new,old} build systems".
- Olga: From the users perspective, libraries will create modules. From the build perspective it isn't a question of how big one module is, but how many modules and how long of a dependency chain do you have. This is what will affect the build throughput the most.
- Bryce: Specific use cases to address?
- Tom: Existing build systems need to be able to consume modules, with minimal updates. Only updating flags.
- Existing build systems:
 - CMake.
 - Make.
 - Boost Build.
 - Internal company build systems.
 - autoconf.
 - Meson.

- ninja.
- shell scripts.
- scon.
- waf.
- Bazel.
- Buck.
- Cargo.
- Gulp.
- WebPack.
- Ant.
- llbuild.
- Evoke.
- msbuild.
- qbs.
- qmake.
- Mathias Stearn: As a user of scon, I don't want scon to support modules so I have yet another reason to move off of it :).
- Corentin: We should say in the TR that waf is not supported.
- Ben B: Isn't waf just a scon fork and stripped down? My only experience is via mpv where it's decent (but also a c-only project).
- Steve D: Use case: Hello world with modules.
- Olga: Just building module interfaces for tooling purposes. Code completion.
- Mathias: Distributed builds (high bandwidth + high latency, and low bandwidth + low latency environments):
- FastBUILD.
- IceCC.
- IncrediBuild.
- sccache.
- ccache.
- distcc.

- Bezel remote build execution.
- My company's internal distributed build systems.
- Mathias: What does an ideal one look like?
- Tom: Static analysis tools, where source code is paramount. Comments get used sometime.
- Coverity.
- Clang static analysis.
- Grammatech (Aaron Ballman).
- CppCheck.
- Ben B: Static analysis: viva64.
- Mathias: Incremental builds.
- Bryce: Dependency scanning vs. explicit module dependency example.
- Side by side example where one version uses scanning, other uses explicit deps.
- Tom: Other tools that traditionally operate on a single TU:
- SWIG.
- clang tooling.
- Qt Moc.
- Test mocking frameworks.
 - Google mocks.
- Bruno Lopes: GCCXML.
- Ben B: Superseded by castXML.
- Ben B: SWIG reads and writes c++; protobuf writes it.
- Mathias: Won't anything using clang-tooling JustWork with modules with something like -frewrite-imports?
- Tom: Mathias, maybe? So long as BMIs aren't shared between tools/compiler built with incompatible Clang versions.
- Mathias: I think -frewrite-imports is a BMI-free solution. Bruno, can you confirm?
- Tom: -frewrite-imports might consume an existing BMI.
- Bruno: It could, but it doesn't need to.
- Mathias: I was thinking of it similarish to nim-lang which compile to C++ source among other targets.

- Ben B: Cocinelle? Though its mostly pattern matching anyways AFAIK.
- Things that generate code (out of scope?):
- protobuf.
- Corentin: Mixed build systems:
- Ex: CMake and QMake in the same build.
- More likely Cmake + lots of automake.
- Olga: Internal distributed builds (MS and others have it).
- Steve D: Internal vs. external modules (things from my project vs. out of my project).
- Steve D: Bug reporting tools, like creduce, delta.
- Bryce: Modularizing libraries:
 - Header only libraries.
 - Boost.
- Ben B: Provide `#includes` for backwards compat.
- Corentin: CUDA, Vulkan, OpenCL, SYCL.

§ 3.2. 2019-03-22 Meeting Minutes

Attendance:

- Anna Gringauze.
- Ben Boeckel.
- Ben Craig.
- Bruno Cardoso Lopes.
- Colby Pike.
- Gor Nishanov.
- JF Bastien.
- Mark Zeren.
- Mathew Woehlke.
- Mathias Stern.
- Michael Spencer.

- Nathan Sidwell.
- Olga Arkhipova.
- Peter Bindels.
- Rene Rivera.
- Steve Downey.
- Stephen Kelly.
- Tom Honermann.

Chair Notes (Bryce Adelstein Leibach):

- Timing: The TR will be feature-driven not deadline-driven; we'll ship it when it's ready.
- Proposed Outline (based on Bryce's pre-Kona notes and Rene/Corentin's Kona slides, see end of document for details).
- Usage: Explains the requirements and expected usage of modules across the C++ ecosystem. Raises questions which need to be addressed later in the document.
 - Stakeholders: Different types of users of modules (more details below).
 - Archetypes: Concrete examples based on expected usage (more details below).
- Findings: Focused technical sections that explore open questions in detail and present the results of field experience. Unopinionated.
 - Module Mapping: What approaches and formats are effective for communicating module name <-> module-interface/header file mappings? Module name + configuration <-> BMI mappings?
 - Module Naming: How should module names be structured? How do we avoid conflicts between different projects? How do we deal with versioning?
 - Module Granularity: What size should modules be to maximize performance and usability? Does the cost of an import scale with the size of the module?
 - Module ABI: How do we maintain stable ABIs in a modular world?
 - Codebase Transition Path: How should projects transition from headers to modules? How should projects support both pre-C++20 headers and C++20 modules?
 - BMI Configuration: How do we find the BMI that was compiled in the same way as the current TU? What defines the configuration of a BMI?
 - BMI Distribution: How effective is the distribution of BMIs alongside module-interface/header files?

- Dependency Scanning: How do we do dependency scanning in a modular world? Can we make it fast?
- Build Performance: How do modules impact build performance? What impact does modules have on parallelism in C++ builds?
- Guidance: Concise set of guidelines for the C++ ecosystem. Addresses questions raised in Usage and draws conclusions based on results from Findings.
- Do we agree to use this outline as a starting point?
- What's missing (Findings sections in particular)? How can this be improved?
- Who is interested in working on a particular section of the proposed outline? Please collect a list of names. Don't volunteer for everything - just for what you care about and can commit to working on.
- Stakeholders:
- C++ Implementations:
 - GCC, Clang/LLVM, Visual Studio, EDG, PGI, ICC, xLC.
- Build Systems:
 - CMake, Make, Boost Build, autoconf, shell scripts, Meson, Ninja, Scons, Waf, Bazel, Buck, Cargo, Gulp, Webpack, Ant, llbuild, Evoke, qmake, MSBuild, internal company build systems, mixed build systems, distributed build systems (icecc, ccache, sccache, incredibuild, distcc, fastbuild, Bazel remote build execution).
- Tools:
 - IDEs, Clang-based tools, CastXML, static analysis tools (Coverity, Clang Static Analyzer, Grammatech, cppcheck), code generation tools (QT Moc, Protobuf), test frameworks (Google Test), test case reduction tools (creduce, delta).
- Libraries:
 - Boost, header only libraries.
- Distributions:
 - vcpkg, conan.io, Linux distributions (RPM-based, Debian-based).
- Other Languages:
 - CUDA, OpenCL/SYCL, C, Python, Rust, Java, SWIG.
- End Users.

- Who is interested in working on a particular stakeholder group? Collect a list of names.
- For each stakeholder group, we need a short description of the group (e.g. what things are in the group) and a bullet list of the issues that matter for that group. Volunteers? Collect a list of names.
- Archetypes:
- Hello world with modules.
- Header only library.
- Incremental build.
- Distributed build.
- Building BMIs only for tooling consumption.
- Dependency scanning vs explicit module dependencies build.
- We need more concrete examples. Who volunteers to go write some up? Collect list of names.
- Format: The TR will likely need to be in Latex, using a fresh fork of the IS Latex that has been customized, similar to the Coroutines TS Latex. We have had very painful issues with non-Latex formal documents in the past. Ex: The Parallelism TS v2 was originally written in HTML which was converted to PDF for publication. It had to be completely rewritten in Latex after we voted to publish it because of typesetting issues raised by ISO that could not be resolved.
- <https://github.com/cpp-tooling> has been created for collaboration. Not sure how we'll use it yet; feel free to create a repository for examples and/or brainstorming. JF Bastien can add people while Bryce is away.
- Appendix.
- List from Bryce's pre-Kona notes.
 - Module Map Format.
 - Name of Module + ABI Hash -> Physical Location.
 - Module Mappers.
 - Module ABI Hashing.
 - Module Versioning.
 - Dependency Scanning.
 - How should tools use a dependency scanner? Command line? Programmatic API?
- List from Rene/Corentin presented at the Kona evening session:
 - Module name <-> Module header unit name mapping.
 - Module name <-> BMI mapping.

- Module naming.
- Guidelines for BMI implementations strategies.
- Guidelines (maybe format) for shipping modularized closed source libraries.
- Guidelines for Linux distributions (maybe).
- Guidelines/format for handling legacy header units.
- Guidelines for using modules:
- ABI concerns/hashing.
- Not authoring modules for 3rd party code.

Chair Notes (Ben Craig):

- Possibly interleave findings and guidance?
- Need concrete code base / code bases, need multiple concrete build systems.
- Tie things to specific examples.
- For multiple stakeholders, use cases.
- Kitchen Sink CopperSpice example?
- Volunteers/interest in particular aspects of the Technical Report:
- Peter Bindels + Gaby: Volunteering for hello world with modules section.
- Corentin Jabot, Peter Bindels, Mathias Stearn: Module Mapping P1484 mapping to source files. Mapping to BMI. Where to find a header unit.
- Michael Spencer and Ben Boeckel: Dependency Scanning.
 - What we've found, and what others might want to do. This is from the impl's perspective.
 - Does the "can we make it fast?" belong in the TR?
- Mathias Stearn + Rene Rivera: Build Performance?
 - Can't get recommendations yet based off of current work.
 - Different models of building? Concurrent BMI and .o vs. distinct builds. Needs research.
 - Chicken and Egg.
- Ben Craig + Tom Honermann + Steve Downey + Stephen Kelly: Codebase Transition Path.
- Olga Arkhipova + Bruno Cardoso Lopes (maybe?): BMI Distribution.
- Mathias + Ben + Googler TBD: Distributed build.
- Microsoft (Gaby + Olga): Dependency scanning vs explicit module dependencies build.

- Michael Spencer: Incremental Builds (information on performance and build theory).
- Anna Gringauze, Tom Honermann: Building BMIs only for tooling consumption:
 - Probably going to look at source. Explain how tools are different from compilers.
 - Sharing BMIs between compilers and tools.
- Bruno Cardoso Lopes (maybe?): BMI Configuration:
- TBD:
 - Header only library.
 - Module Naming.
 - Module Granularity.
- Stakeholders to be covered on a per item basis.
- Hope that another meeting we can establish priorities.

Minutes (Tom Honermann):

- Ben C: Introduces agenda; agreement on the outline of the TR; get volunteers; minimal tech talk.
- Ben C: Any objection to the findings as an outline?
- JF: Need to base this on code at some point. Would like to see a code base that uses modules and multiple build systems. We can talk all day about theoretical concerns, but need to base work on reality.
- Ben C: As an example, modularize Boost?
- JF: Need to focus on applications.
- Ben C: Need to distinguish modularizing a library and consuming a library.
- Peter: Perhaps try the kitchen sink example from CopperSpice?
- Tom: JF, do you want real projects or exemplary projects?
- JF: Real projects.
- Steve D: Don't think we need really real projects, just exemplary ones. POSIX demonstrates how to do compiles, link; use case oriented.
- Tom: Agree, and it would be nice to have examples in the TR demonstrating usage.
- Ben C: Bryce has a hello world with modules that could be an example in the TR.
- Peter: I volunteer to make a hello world example.

- Ben C: Back to the outline, who is working on what? Corentin and Peter are working on module mapping?
- Peter: We have P1484.
- Mathias: I volunteer to help with module mapping.
- Tom: Which aspect of module mapping are we discussing here?
- Peter: Mapping to source.
- Mathias: Also want mapping to BMIs.
- Ben C: Sounds like this covers mapping to source, BMI, and indication of header units.
- Ben C: Michael Spencer is working on dependency scanning. So is Ben Boeckel. Can I record them as volunteers to work on this?
- Michael: Yes.
- JF: Dependency scanning is part of build system implementation. What is the goal of discussing dependency scanning (and other features we're discussing) as part of the TR?
- Mathias: It is a contract between stakeholders.
- Tom: Not concerned about implementation details; concerned about ensuring meta data is represented in ways usable by multiple tools, build systems, etc..
- Michael: The best/fastest way to build modules are relevant for implementation.
- Peter: Perhaps worth discussing trade offs between fast and accurate?
- Mathias: Let's take inaccurate off the table.
- Ben C: Moving on to build performance, can I sign Rene and Mathias up for that?
- Mathias: Yes, questions to address: can BMI and object files be built concurrently? What gets built and how? These are worth researching.
- Rene: Happy to work on performance related issues and testing. There is a chicken/egg problem of needing working compilers. We can't tackle distributed builds without additional work.
- Mathias: Would like feedback on how reasonable it is to look at performance of current compiler incarnations.
- Nathan: In three years time, performance profile will probably be quite different; focus now is correctness, not speed.
- Michael: Clang has some inefficiencies around finding modules now. I think overhead of modules will go down over time. Dependencies will remain.
- Mathias: Wondering about relative performance, scanning vs code gen vs BMI gen, etc... Perhaps a TR2 would be a good focus for performance.

- Michael: Some performance sensitive things will change, some things won't.
- Ben C: Looking at code base transition path now. BC volunteers.
- Tom: Interested in transition path.
- Stephen K: Also interested.
- Steve D: Also interested. Will bring a Lakos and Bloomberg informed focus.
- Ben C: Olga, will you sign up for BMI distribution?
- Olga: Yes. We've had internal discussions about sharing BMIs.
- Ben CL: Volunteers to work on BMI distribution as well.
- Steve D: This overlaps with sharing of object files as well. What is the range of IFNDR when sharing BMIs? If a BMI isn't suitable, how does it get recompiled.
- Tom: Is Michael interested in volunteering with regard to BMI distribution?
- JF: We can volunteer to write a section that says "don't".
- Ben C: I don't think we should have a section that raises questions for all stakeholders. Instead, each area under findings should raise questions and explore them from the standpoint of each stakeholder.
- Mathias: I agree, though not particularly productive to discuss now until we have stuff to put in the doc.
- Ben C: Makes sense.
- Ben C: Peter volunteered for hello world, volunteers for distributed build?
- Mathias: I volunteer. Would be nice to have someone from Google due to difference in approaches.
- Gor: Volunteers Gaby to contribute to hello world examples.
- Ben C: Would like to sign up Gaby for explicit module dependencies as the Microsoft Edge team purportedly used them.
- Mathias: Do we want to encourage explicit module dependencies?
- ??: No.
- Tom: Matches existing PCH usage in Microsoft ecosystems.
- Ben C: We should discuss.
- Olga: For dependency scanning, we are planning to do work to support this, but haven't started yet. Mixed mode dependency scanning and explicit dependencies may happen.

- Ben C: Moving on to header only libraries. Done today to avoid build system pain. Any volunteers?
- Peter: Catch2 considering moving away from header-only for technical reasons (e.g., build speed).
- Ben C: No volunteers for header-only.
- Ben C: Volunteers for incremental-build? Kind of inherent to builds in general...
- Mathias: I volunteer to writeup something for incremental builds.
- Ben C: On to building BMIs for tooling consumption.
- Olga: I work on static analysis, so interested in special tools. Interested in saving information useful for tools in BMIs.
- Tom: Will contribute to discussion on sharing BMIs across compilers/tools.
- Ben C: No assignments for module naming, module granularity, BMI configuration. Got the rest. Stakeholders to be covered on a per item basis.
- Tom: Perhaps next meeting we can have everyone vote about their highest priority concerns to be addressed in the TR.

§ 3.3. 2019-04-05 Meeting Minutes

Attendance:

- Ben Craig.
- Bryce Lelbach.
- JF Bastien.
- Ben Boeckel.
- Bruno Lopes.
- Gabriel Dos Reis [Gaby] (Microsoft).
- Isabella Muerte [Izzy].
- JF Bastien.
- Mark Zeren.
- Mathias Stearn.
- Rene Rivera.
- Steve Downey.

- Micheal Spencer.
- Olga Arhipova (Microsoft).

Chair Notes (Bryce Adelstein LeBach):

- Timing: The TR will be feature-driven not deadline-driven; we'll ship it when it's ready.
- Proposed Outline (based on Bryce's pre-Kona notes and Rene/Corentin's Kona slides, see end of document for details).
- Usage: Explains the requirements and expected usage of modules across the C++ ecosystem. Raises questions which need to be addressed later in the document.
 - Stakeholders: Different types of users of modules (more details below).
 - Archetypes: Concrete examples based on expected usage (more details below).
- Findings: Focused technical sections that explore open questions in detail and present the results of field experience. Unopinionated.
 - Module Mapping (Nathan Sidwell, Corentin Jabot, Peter Bindels, Mathias Stearn): What approaches and formats are effective for communicating module name <-> module-interface/header file mappings? Module name + configuration <-> BMI mappings?
 - Module Naming: How should module names be structured? How do we avoid conflicts between different projects? How do we deal with versioning?
 - Module ABI: How do we maintain stable ABIs in a modular world?
 - Codebase Transition Path (Ben Craig, Tom Honermann, Steve Downey, Stephen Kelly): How should projects transition from headers to modules? How should projects support both pre-C++20 headers and C++20 modules?
 - BMI Configuration (Bruno Cardoso Lopes): How do we find the BMI that was compiled in the same way as the current TU? What defines the configuration of a BMI?
 - BMI Distribution (Olga Arkhipova, JF Bastien, Michael Spencer): How effective is the distribution of BMIs alongside module-interface/header files?
 - Dependency Scanning (Michael Spencer, Ben Boeckel): How do we do dependency scanning in a modular world? Can we make it fast?
 - Build Performance (Mathias Stearn, Rene Rivera): How do modules impact build performance? What impact does modules have on parallelism in C++ builds?
 - Module Granularity: What size should modules be to maximize performance and usability? Does the cost of an import scale with the size of the module?

- Guidance: Concise set of guidelines for the C++ ecosystem. Addresses questions raised in Usage and draws conclusions based on results from Findings.
- Stakeholders:
- C++ Implementations:
 - GCC, Clang/LLVM, Visual Studio, EDG, PGI, ICC, xLC.
- Build Systems:
 - CMake, Make, Boost Build, autoconf, shell scripts, Meson, Ninja, Scons, Waf, Bazel, Buck, Cargo, Gulp, Webpack, Ant, Ibuild, Evoke, qmake, MSBuild, internal company build systems, mixed build systems, distributed build systems (icecc, ccache, sccache, incredibuild, distcc, fastbuild, Bazel remote build execution).
- Tools:
 - IDEs, Clang-based tools, CastXML, static analysis tools (Coverity, Clang Static Analyzer, Grammatech, cppcheck), code generation tools (QT Moc, Protobuf), test frameworks (Google Test), test case reduction tools (creduce, delta).
- Libraries:
 - Boost, header only libraries.
- Distributions:
 - vcpkg, conan.io, Linux distributions (RPM-based, Debian-based).
- Other Languages:
 - CUDA, OpenCL/SYCL, C, Python, Rust, Java, SWIG.
- End Users.
- Archetypes:
 - Hello world with modules (Peter Bindels, Bryce Adelstein Lelbach).
 - Header only library (Bryce Adelstein Lelbach).
 - Incremental build (Michael Spencer).
 - Distributed build (Mathias Stearn, Ben Craig, Manuel Klimek?).
 - Building BMIs only for tooling consumption (Tom Honermann, Anna Gringauze, Olga Arkhipova).
 - Dependency scanning vs explicit module dependencies build (GDR).

Minutes (Ben Craig):

- Ben C: Outcome for prioritization is unclear.
- Bryce: Things without volunteers are implicitly lower priority.
- Mathias: Module naming and granularity could be folded into other things.
- Bryce: At one point, it was combined.
- Gaby: That's more of a coding guidelines, not clear to me that it belongs here..
- Mathias: It does have performance tradeoffs, particularly when switching between library sized vs. header sized modules..
- Izzy: This should be for general guidelines, because we could probably convince (for example) the Bloomberg people that it should be one class per partition vs. one class per module.
- Gaby: Hoping the TR isn't just a set of guidelines. It can't have the force of a standard, but should be more than just a guideline.
- Izzy: Maybe put something in the core guidelines for granularity.
- Mathias: Should have data to back that up.
- Bryce: That's why the section is called "findings". Should be based on experience and data.
- Bryce: Move Module ABI to transition path?
- Steve D: Not just part of the transition path.
- Bryce: Should have clearly identified people / reps for the stakeholders.
- Bryce: Volunteering to work on header only libraries.
- Nathan Sidwell's P1602R0 was presented in Kona in EWG, may still get presented here.
- Ben B: Output format to describe what a source file uses and produces.
- Bryce: What would this look like if import.cpp was a module partition.
- Ben B: (adds some colons to file path) Depends on what gcc outputs for the actual partition BMIs. Haven't tested that yet. This doesn't care about is-a-partition or not.
- Mathias: What does this return for pure implementation units that are not importable?
- Ben B: Wouldn't be any provides.
- Mathias: Provides arrays might be useful if we come up with a multi-module format for distribution.
- Gaby: Just a matter of time before we can make multiple modules from one file in C++.
- Mathias: Nathan Sidwell is looking to provide extensions along these lines.

- Bryce: This doesn't mention the BMIs configuration.
- Ben B: It does not.
- Gaby: This is the output of the scanning phase? And it could be input to a build definition?
- Ben B: It could be.
- Gaby: Just trying to be clear on which phase of the build this comes up.
- Mathias: Concerned that the compiler is telling the consumer where files are going, rather than the other way around.
- Ben B: The directory came from an initial compiler invocation string.
- Mathias: Why not just use logical names.
- Ben B: Because without a module map, gcc just comes up with these locations. So the JSON just has a file path hint.
- Mathias: so these names are just suggestions.
- Ben C: Not fond of the path mismatches where the json gives a path that may not be used.
- Ben B: things like filepath are optional.
- Mathias: Why include optional things?
- Ben B: This is why you need a collator.
- Mathias: Then let the collator do the logical to physical mapping.
- Mathias: What does it mean for a compiler to want a path?
- Steve D: It's based off of a compiler flag or the compiler defaults.
- Mathias: Since you're going to need to do the mapping, I'm not sure why you want the filepath in the json.
- Ben B: Will investigate omitting filepath.
- Steve D: If you don't tell the compiler something, it will pick something.
- Izzy: Letting the compiler pick file locations made it very difficult to build things in a performant way. Had to do lots of docker work to fix that in a past project.
- Gaby: It's not a design flaw in GCC that it outputs things by default.
- Olga: Traditionally all build systems are explicit about the outputs.
- Olga: Would like to focus on the "provides" and "requires". Don't want "depends" to be required. Want a special mode that has less information for performance.
- Ben B: flag for minimal information would be fine, but that file wouldn't be useful for some build tools.

- Mathias: Why would a build tool need anything besides provides and requires.
- Ben B: Need to provide dep files as an input edge.
- Bryce: Is there any existing thing for fortran?
- Ben B: No. Cmake has to parse fortran today.
- Bryce: Was Olga's concern just performance?
- Olga: Yes.
- Bryce: Without data, I'm hesitant to raise this as an issue.
- Olga: Not just planning on using this for builds, but other operations like IDEs. Speed is really critical there since it is interactive.
- Gaby: Have you seen how long it takes to output the information?
- Ben B: No, haven't measured performance yet.
- Olga: It's not just producing the json, but parsing the source to get that information.
- Ben B: Note that scanning doesn't need to do full preprocessing.
- Spencer: In a few days I'll be giving a talk on dep scanning for modules. Scanning then doing an explicit build is faster than doing an implicit build.
- Gaby: Would like a specification as to what is meant when we say scanning.
- Bryce: Different applications may need different amounts of scanning.
- Ben C: Getting a skeleton spec is blocking things.
- Izzy: How does the JSON handle unicode?
- Ben B: You end up with integer encoded things, and URL percent escaping.
- Izzy: Considered using base 64 encoding?
- Ben B: It could be done, but you need to know endianness.
- Mathias: Don't need to support non-native endianness.
- Ben B: No endianness if you are outputting ascii integers.
- Bryce: Getting a skeleton repo set up, Latex is basically going to be required.
- Izzy: There are tools that can emit both.
- Rene: Having graphs in Latex is going to be challenging.
- Bryce: Amount of startup overhead is a concern. We can reuse what the standard is already using.
- Bryce: I have used restructured text recently, maybe that is an option.
- Bryce: Volunteering Ben C and Gaby on helping with Github. Izzy for RST things.

- Gaby: Can we reuse the C++ github?
- Bryce: Will check with Herb.

§ 3.4. 2019-04-12 Meeting Minutes

Attendance:

- Bryce Adelstein Lelbach.
- Tom Honermann.
- Nathan Sidwell.
- Anna Gringauze.
- Ben Boeckel.
- Ben Craig.
- Bruno Lopes.
- Isabella Muerte [Izzy].
- JF Bastien.
- Michael Spencer.
- Mathias Stearn.
- Przemyslaw Walkowiak.
- Rene Rivera.
- Steve Downey.

Minutes (Ben Craig):

- Bryce: New mailing list should be up and running next week, archives may not be migrated.
- Nathan presents P1184.
- Bryce: What is "cookie".
- Nathan: Just an identifier of which compilation / client is calling. For files, also a way of saying which lines to pay attention to. Not a strong crypto cookie. Implementation and name has changed.
- Nathan presents P1602.
- Bryce: How would you end up with a cycle with modules?

- Nathan: An erroneous program. This would be detected here, and could report on that problem. This prevents Make deadlocks.
- Nathan: The module mapper is part of make. Defining the magic variable is how make knows what to talk to.
- Bryce: Does make do anything for fortran modules or similar languages?
- Nathan: I don't think make has any intrinsic smarts to deal with module systems.
- Nathan: Discovering dependencies requires you to build dependencies during the discovery.
- Ben B: Historically, for Fortran, the answer has been "run make until it works". In 2008, Fortran support was added to cmake for the makefile generator. It's supportable with POSIX make. Need something to do the scanning. Compilers historically write modules where the modules want, without letting the build system tell it where to put things. We're going to push to have fortran support the same build system format description. Brad King is the maintainer of the fortran support.
- Tom: GNU make supporting rules for multiple compilers seems tricky.
- Nathan: Do you want to build some parts in compiler 1 and others in compiler 2?
- Tom: Yes.
- Nathan: How does make deal with it?
- Tom: It doesn't, you need to write your own rules.
- Bryce: My understanding is that the module mapper is not responsible for dealing with BMIs of different configurations, the user is. If I have some gcc builds and some clang builds, the BMIs aren't compatible. Would I need different mappers, different cookies, different servers?
- Nathan: Similar to debug vs. release? Canonical way is to have different directories for different configurations. Put the different BMIs in different directories. You would have multiple mapper services.
- Mathias: Don't want to require people use different directories and let ccache handle it. Otherwise it gets exponentially explosive.
- Bryce: Not sure how that would work.
- Mathias: Put everything in one directory, let ccache switch between configs.
- Izzy: ccache doesn't just look at file contents, also looks at command lines.
- Ben B: How does ccache work with split dwarf today?
- Mathias: ccache just knows about dwarf. May not know about other things.
- Ben B: So ccache basically knows how to parse gcc flags too?

- Izzy: It does not. It hashes the command line after sorting the argv.
- Ben B: Hrm, flags can't be sorted like that.
- Izzy: I might have misread the github issue, i'll have to go back and check. but they do go out of their way to avoid hashing the file as much as possible.
- Mathias: No it definitely knows about the flags, as does icecc.
<https://github.com/ccache/ccache/blob/master/src/ccache.c#L2656> and
<https://github.com/icecc/icecream/blob/master/client/arg.cpp#L444>
- Bryce: We should discuss ccache and sccache in a future meeting.
- Tom: Depending on the build system not just to do module mapping, but also dependencies. Doing that based on what's encoded in the makefile. How do we fit this into tools for ides or code generation tools that need to resolve dependencies.
- Nathan: Dependencies aren't encoded in the makefile.
- Tom: Build system needs to tell the compiler where to put the BMI.
- Ben C: Modules may not create new classifications of problem areas, but it puts a lot more things in a bucket that used to be pretty small. Sort of a union of code generation header files and linking.
- Nathan: I somewhat address that in the presentation. Easy to conflate a lot of different problems. Need to be sure we distinguish between the new problems and the old problems.
- Bryce: Today, make doesn't have any language specific knowledge. How did you end up with that design?
- Nathan: needs to interact with the build graph construction, and that required new work. You need to be inside make for that.
- Tom: Header units and determining when something should be included vs. imported. Did you experiment with that?
- Nathan: Yes, needed to put some bits in the Makefile for that to tell everything that "legacy.h" is a header unit.
- Tom: Header file has to be consistently translated.
- Mathias: Standard calls out if a header is importable, you must transform it to an import.
- Tom: So that information has to be provided extrinsically.
- Steve: Even more subtle in the standard, because there is implementation defined behavior in there.
- Mathias: But things that are in that set must be consistent.
- Mathias: Would a pragma at the top of the header to say that this is importable make sense?

- Nathan: That may have been discussed on the mailing? Pragma that could say this is importable. Also a pragma to say that "foo" is importable so that one wrapper header could annotate lots of things. `include_next` kind of scheme could also be used with shim headers.
- JF: `#pragma once`.
- Tom: Not `#pragma once` because the difference between `include/import` is observable (in some cases).
- Mathias: <http://eel.is/c++draft/cpp.include#7> for the curious.
- Nathan: The set of importable headers is not deducable, but everything else should be deducable.

§ 3.5. 2019-04-26 Meeting Minutes

Attendance:

- Bryce Adelstein Lelbach.
- Ben Boeckel.
- Ben Craig.
- Bruno Lopes.
- Corentin Jabot.
- Isabella Muerte [Izzy].
- JF Bastien.
- Jayesh Badwalk.
- Mark Zeren.
- Mathian Stearn.
- Mathias Stern.
- Michael Spencer.
- Olga Arhipova (Microsoft).
- Rene Rivera.
- Richard Smith.
- Steve Downey.
- Tom Honermann.

Minutes (Bryce Adelstein Lelbach, First Half):

- Ben starts presentation on distributed build systems.
- Bryce: What type of preprocessing does distcc do? Does it just concat in macros? What about `#includes`, which can have macro expansions in them?
- Ben, Mathias: It's clever.
- Bryce: So you're saying that FASTBuild might be able to be made to work out of the box, using the support for distribution of extra files?
- Ben: Maybe, although I wouldn't recommend it.
- Tom: Do you have any information on the prevalence of the two different distcc modes?
- Ben: Not really. Pump mode is newer.
- Tom: How easy is it to switch between the two?
- Ben: You have to teach it about your build system. It's non-trivial.
- Ben: Pump mode is potentially faster.
- Bryce: How could we ever make FASTBuild and normal distcc mode work? They expect a single file to contain a single translation unit. Modules is explicitly trying to move us away from textual inclusion.
- Ben, Mathias: Not necessarily. Compilers might be able to support this (with things like `-frewrite-imports`).
- Richard: We support `-frewrite-imports` currently only for header units, but there's no reason that we couldn't make it work for all modules.
- Bryce: Is this a bad model for these tools to use?
- Richard: You're probably using a distributed build system because you care about build system. With this model, you're taking a small file and some build artifacts and turning them into larger files. You might be giving up performance. But from a semantic perspective, this should work.
- Ben: This might be useful as a deployment/migration tool, though.
- Tom: Is it reasonable to expect all implementations to have something like `-frewrite-imports`?
- Richard: It requires certain properties of your BMI file. You must be able to take the info in a BMI file and reconstruct source from and then reprocess.
- Richard: Another idea would be to have the compiler build you a package for a particular TU.
- Tom: But then you might end up shipping a lot more text than you need to.
- Richard: You could take a hybrid approach, where you preprocess for directives only, and also package up BMIs.

- Ben: The packaging approach doesn't work as well for things like static analysis and creduce. Those tools want to be able to see all the source code.
- Mathias: Can you package up the textual source of the module interface units instead of the BMI?
- Richard: Maybe, but then you need to have a mapping to file names.
- Olga: Modules can be built by different projects/targets with completely different command lines. Just using the sources and hoping that the command lines match up is probably not a good idea.
- Mathias: Don't we have that problem either way?
- Mathias: There was an assumption in what I said that the build flags were part of the module <-> file mapping (e.g. module <-> file + build flags).
- Izzy: How is this going to work with module partitions? This seems like it will be a big pain.
- Ben: How will that be difficult?
- Richard: Module partitions for the purposes of distribution behave similarly to using different modules, so it's not necessarily harder.
- Tom: How feasible is it to only distribute the BMIs? Do you also need the corresponding sources? I've heard that some implementations need the source in addition to the BMI, for things like diagnostics, etc.
- Richard: That is the case for Clang. However, Clang has a mode that can embed the source into the BMI.
- Bryce: Is the source needed for things to work, or just for diagnostics?
- Richard: It's need for things to work; if you don't have it, things will break.
- Izzy: What about std::embed?
- Ben C: FYI, I believe Jean-Hyde is no longer pursuing std::embed.
- Izzy: He and I were discussing it on the include cpp discord yesterday. It might get split up into two separate bits, but he is a bit swamped and is not going to pursue it for a bit. We might see it in 2023/2026.
- Mathias: Shipping BMIs in this mode where you make a single file per TU seems difficult. I believe BMIs are 5-10 larger than source and are less compressible. If you're network bound, that'll be a problem.
- Richard: In my experience, they are about 3x larger and are less compressible.

Minutes (Ben Craig, Second Half):

- Richard Smith presents on clang modules and module maps.
- Bryce: Module maps that clang supports are just for header units, right?

- Richard: Yes.
- Bryce: Will you extend that format for other modules?
- Richard: Not been my intention to do so. Module maps work well for mapping from name of header file to a module. If you are going from module name to source file, that is harder because you need to read a lot of files on the file system to find that. Not sure that is scalable. Would be better if the compiler is more directly told about this.
- Tom: So this is a lesson learned from objective-C.
- Richard: Yes. But there are other problems too, like finding multiple conflicting maps in projects that wouldn't be used together anyway.
- Bryce: This is similar to include directories today?
- Richard: This is worse, because it doesn't even work for when everything is in /usr/include, because you need to recurse down.
- Tom: Only get one module map per directory? This is a problem for packaging.
- Richard: I can imagine a packager managing the module map for you.
- Tom: Or having a libfoo.module.map.
- Mathias: Is a "master" module map not suitable for objective C.
- Richard: Objective C solves this a different way, but it may work for C++ if you can guarantee uniqueness across the system.
- Bryce: What is clang planning on doing for mapping module names to source.
- Richard: Most straightforward thing is to have the build system pass in the list of inputs directly on the compile command line. If you want something more implicit...
- Bryce: Google has been doing this at large scale and it's been working, you don't need something different?
- Richard: We started with implicit mode. It was convenient, but didn't distribute well and didn't scale well. More sophisticated build systems are going to want to have more control.
- Tom: My concern is with things that aren't using the build system.
- Bruno: We pay a high price for the downward recursive search in objective C. We don't want to encourage that if we can help it. Our mac frameworks aren't a problem, but other, non-framework locations are an issue. We did a downward search initially, and we're stuck with that approach.
- Bryce: Richard and Nathan started with implicit systems. Our first users didn't like it, and had us move to something more explicit. Field experience is guiding us to explicit models.
- Tom: We aren't getting a lot of data there. Coverity data says that a lot of people are using implicit modes with xcode.

- Corentin: Mapping of name to bmi isn't important, because build system knows it exists, and shouldn't be implicit. Mapping of module name to source name is more valuable though.
- Bruno: The most valuable part is that it is easy for users to do things, but it has scaling issues. If you want to scale, you will need to change models.
- Tom: Do we need an implicit model to have success? I think we do.
- Ben B: If you do -fmodule-map=file, it will look in there both for reading and writing BMIs.
- Bryce: Any reason compilers couldn't expose multiple strategies for module mapping?
- Richard: Clang already does that. Don't know of a technical reason other than adding more complexity to compiler.
- Bryce: Does the client / server mapper have any appeal for your use cases?
- Richard: If we were to do that, main reason would be compatibility with GCC. It's nice in that it decouples concerns. Seems like a fine approach.
- Richard: In my experience, they are about 3x larger and are less compressible.

§ 3.6. 2019-05-10 Meeting Minutes

Attendance:

- Ben Craig.
- Ben Boeckel.
- Gabriel Dos Reis [Gaby] (Microsoft).
- JF Bastien.
- Lukasz Menakiewicz (Microsoft).
- Olga Arhipova (Microsoft).
- Mark Zeren.
- Nathan Sidwell.
- Tom Honermann.
- Corentin Jabot.
- David Blaikie.
- Richard Smith.

Minutes (Ben Craig):

- Corentin presenting his proposal on module naming:
<https://isocpp.org/files/papers/D1634R0.html>
- Tom: What's the status quo on if we have a conflict?
- Nathan: Ill-formed, only when you try to import the wrong one.
- Corentin: Build system can give diagnostics if you have two modules with the same name in the same dependency graph.
- Gaby: You can have conflicts within an organization.
- Nathan: We also have this collision problem with namespaces. That doesn't seem to be a problem that requires vendor specific namespaces. Why do we think this will be a bigger problem with modules?
- Ben B: One thing we do with our third party packages is mangle the packages. We change the names of the shared library and of the dynamic symbols.
- Nathan: There must be some subtleties there.
- Tom Honermann: Namespaces merge naturally.
- Ben C: You need a double collision (namespace and class names).
- Mark Zeren: These collisions are a problem. Google allegedly has a registry for top level namespaces. VMWare probably needs one.
- Sidwell: Would like to see that kind of rationale in the paper, to illustrate that it is a problem.
- Tom: When we import a module, we need the unique file that corresponds to the module.
- Gaby: Difference is that with namespaces, we get collision of members, and it can be subtle.
- Ben B: <https://github.com/mathstuf/cxx-modules-sandbox/blob/master/link-use-mask/CMakeLists.txt>.
- Nathan: There seem to be some mitigating strategies today, but we aren't describing today's mitigations. Why don't we already hit this problem with existing software. On most CPP projects, the dependency tree is shallow, so they mostly conflict with themselves. At the namespace level, most projects use namespaces as a way to scope names. With headers you can rename the headers on disk to solve some uniquification problems. Two libraries (simple and duplicate) which have a module m. Two executables that import m. First one found is it. Cmake could say that it sees two "m"s and error out. Only works when the same name module is visible in the same place. Two libraries that make a config module way down deep, that is going to be harder to diagnose.
- Corentin: Would be nice to have a way to map module names to file names at some point, and we are limited in what we can put in a file name. Limiting ourselves to the basic character set helps that. We can also avoid some issues if we stick with lower case, particularly when sharing between Windows and Linux.

- Gaby: Other languages have a mix of pascal case vs. lower case. I would like to be able to visually separate module names from namespace names, which would argue for pascal case. I'm not sure if we want to force it, or if this is just a guideline.
- Corentin: There is no enforcement, no language changes.
- Gaby: For the TR, we are aiming for something that everyone would be using and doing. My organization is using a different scheme already, and I would like to avoid that if possible.
- Tom: Are they different just because of the casing, or are they using characters outside of the basic character set?
- Gaby: Just casing.
- Tom: Please switch from ASCII to basic source character set.
- JF: ASCII seems like an overly protective regression.
- JF: I think we should try to fix some of the existing identifiers with unicode. We should fix C++ as a whole, and make modules follow that. I don't think we should race to the bottom and support only the lowest common denominator. We don't want to go back to the 8 character file length. Accept that on some file systems, the mapping will be difficult. What do git and SVN do.
- Gaby: Agree. Misconceptions on Windows. Don't want to base what we are doing on Windows misconceptions.
- JF: We should guide people to names that don't clash.
- Tom: I would not like to limit to ASCII either. Would like to give an algorithm to let tools implement the mapping.
- Corentin: Underlying filename, you can't rename it. You need to have the same mapping on each platform. Because of that, I think we should have a subset of characters. It's about understanding the filename and make sure that tools like cmake will be able to find and name the BMI appropriately.
- Tom: Not sure why the source filename needs to be renamed. Doesn't need to be a correlation between bmi, source, and object name.
- Corentin: Would like to make it possible to have a mapping between the module name and a source file name.
- David: The compiler/build system could implement a name->filesystem name mapping if the filesystem has naming limitations.
- Tom: Would be useful to implement a mapping algorithm (note taker: like punicode).
- Ben: Is there an expectation that if you have a foo.bar, that you also have a foo?

- Corentin: No. If you have `google.abseil`, you don't need `google`. If you `google.abseil.list`, then `google.abseil` should also bring `google.abseil.list`.
- Gaby: This has problems at scale. I could have a subgroup that conflicts with the bigger group. This introduces problems.
- Tom: Should be with something like MISRA, and not the TR.
- Corentin: We want this to make it easier to do dependency management at scale. Can't have name conflicts, as the whole world is your dependency graph. If you have a module name, you could have a global index of every module name that exists, and find the library / project where it is declared and automatically download that library.
- Gaby: If you want the library downloaded with no input, then yes. In practice, you want input from the user. It's ok to have several repos competing to provide something and you have to choose them.
- Corentin: If you have two third party libraries that both import a library named `foo` that you have a conflict that you have no control over. This is one of the reasons you need unique names. The number of dependencies you have can grow exponentially. It needs to not happen to begin with.
- David: Even with unique module names - you still have the version problem. Your dependency A depends on X1, but dependency B depends on X2.
- Gaby: You get to choose in practice, because if you don't, you are in big trouble. There are things like licences that users have to pick.
- Corentin: Renaming a module is a major breaking change.

§ 3.7. 2019-05-24 Meeting Minutes

Attendance:

- Ben Craig.
- Bryce Lebach.
- Nathan Sidwell.
- David Blaikie.
- Rene Rivera.
- Olga Arhipova (Microsoft).
- Tom Honermann.
- Matthew Woehlke.

- Mathias Stearn.
- Stephen Kelly.
- Gabriel Dos Reis [Gaby] (Microsoft).
- Richard Smith.
- Lukas (Microsoft).
- Mark Zeren.
- Michael Spencer.

Minutes (Ben Craig):

- Bryce: sg15@lists.isocpp.org is the new mailing list:
<http://lists.isocpp.org/mailman/listinfo.cgi/sg15>
- Bryce: Started calling BMIs CMI for compiled module interface rather than BMI. Don't want to imply a standardized binary interface.
- Tom: Used term "Artifact".
- Bryce: Would like to talk about file extensions. Maybe different extension for modular implementation units vs. other units.
- Olga presenting on BMI distribution: <http://www.open-std.org/pipermail/tooling/2019-May/000656.html>
- Tom: Some platforms provide PCHs, but they are generally as a fallback.
- Mathias: You already need to have a global and a local index to provide things like find all references or go to implementation? Seems like that would require putting the modules in that format rather than putting things in the codegen format. You probably need different information in each? Seems like a codegen compiler may choose not to put any body information in the bmi if no optimizations are requested. But an IDE would need that for find all references.
- Lukasz: We do have both kinds of indexes. If a user is able to copy a BMI to the system, then we would want to serve intellisense with that as well. Would be jarring not to have that information.
- Mathias: This is presupposing that you wouldn't have source available for a given BMI, and that BMIs would be more than just a cached artifact.
- Lukasz: Wouldn't go so far to say that it is presupposing.
- Mathias: Didn't quite mean that.
- Lukasz: But we do want to have a close parallel between Intellisense and build.
- Gaby: For the Microsoft pre-distributed BMIs, we always provide the source as well.

- Richard: Clang has a language server protocol (LSP). If you ask the build compiler questions through LSP, you avoid cross compiler requirements.
- Mathias: LSP as the main communication mechanism has me concerned. Even in the clang tooling world, there are three different protocols. Would be odd if those needed to use LSP then talk to another compiler,.
- Lukasz: My belief is that LSP is not suitable for this purpose. It is too high level to ask about the compiler artifacts. It's an IDE level interface. Like give me a member list at this location. Current LSP messages probably aren't useful here.
- Richard: The only use case I was suggesting LSP for was the IDE use case.
- Olga: The intellisense compilation is more fault tolerant than the build compiler. The build compiler typically stops parsing as soon as it hits errors. We also have tag parsers that need to extract information from already built BMIs.
- Bryce: What's a tag parser?
- Lukasz: Different optimization approaches for build compiler vs. intellisense. Throughput vs. responsiveness. Engineering cost wise, it is harder to use one compiler for everything.
- Olga: using portion of MSVC compiler for intellisense until 2010, and it was painful.
- Richard: This does resonate with Mathias's concerns, that sharing a file format may not make sense between the different concerns.
- Gaby: The scenario isn't that they want to reuse the contents of the BMI in the way the codegen would, but they do want to look at the structures and idioms that are exported. Suspect they are looking for ways to extract that from the BMI. The expectation is that this information is in all the formats. They are just trying to get at this precomputed information.
- Richard: That's a useful perspective. To a certain extent, it's something more similar to LSP style analysis, maybe that's the wrong model for this though. Some way that not every tool needs to know about every other tools format.
- Tom: For Coverity, the content and the structure is very compiler specific. We use one front end to emulate lots of compiler versions. BMI distribution being compiler version specific is tough for us because we are a floating compiler version. We'd need stability across versions. If we could extract things like the command line and source files used to make the BMI, then that would be useful for us, so that we could make a compatible invocation?
- Bryce: Have you talked to EDG?
- Tom: They said they will support modules, but we don't have more specific details than that.
- Tom: Also unsure about sharing information across clang and gcc. Lots of implementation dependent information than can get encoded. Not sure how that information can be made

compatible across compilers. How does one compiler consume an intrinsic from another.

- Gaby: Primary goal from Olga and Lukasz is to have a mechanism to extract already computed information. Not a goal to share a format that is shared. If you want to go in that direction, then we aren't equipped for that yet. Being able to invoke the compiler or a compiler provided tool to extract that information may be enough. Just need to define what that thing is, may be a tool.
- Bryce: Is there some common set of queries that are useful that everyone wants to ask of a BMI that all implementations can provide?
- Ben: Agree with the goal, highly skeptical of it being achievable. `__Intellisense__`, static analysis macros, `_MSC_VER`, all of those things can make sharing BMIs harder. Dropping those macros would be a big step, but keeping them makes things very difficult.
- Olga presenting again.
- Ben: ABI compatible isn't necessarily object compatible. Borland used to have OMF files vs. COFF files.
- Tom: Clang attempts to make a determination if a cached module can satisfy a request. Replicating that across many tools would be very hard.
- Olga: If we can ask the compiler if you can use the BMI or not, then the knowledge is in one spot and the build system can use it.
- Tom: But the build system would have to ask that question for every possible set of options.
- Olga: When I'm building a .cpp with a module, if I can check to see if the module is applicable, then I can either use the module, or rebuild on demand. Still theoretical.
- Tom: Right, that's my point. The options are even order dependent.
- Olga: Can see if we can relax the requirements on having the exact command line.
- Tom: History of a similar requirement in Coverity. Given an external compiler invocation, we translate to our invocation. Our finding is that it is too difficult to make the translation, and we end up instantiating things repeatedly.
- Ben: You can cheat and just trust the user, like with static libs. For example, I provide debug/release + static/dll versions of boost.
- Bryce: Bruno and Michael will present on this. Compiled modules are more sensitive than object files or static archives would be.
- Michael: We have a potential solution for reducing the number of configurations due to warnings.
- Richard: We have two scenarios. One where we have implicit modules. Here we use the flags that match what are being used. If we have an exact match, we can use a cached version. There's a complication there based off of warning flags. Reduced warning are usually ok. Second scenario is where you are explicitly building module artifacts. When you use them, we are more

permissive in config changes. It's ok to have different predefined macros or include paths there. We don't allow config flags to differ if they would result in functionally different ASTs in ways that matter to us (like language modes). So we do allow some amount of deviation, but only fairly limited. May be enough to help with some of these scenarios.

- Tom: Add to your list, need to know the current working directory and env of the compiler when invoked. Those are the kinds of things that we tend to capture when analyzing a build.
- Olga: Right, those should be part of the "options" and not the "switches".
- Gaby: Please send that list to the mailing list.
- David: you still have two tools parsing the code. Sometimes you gather everything with a compiler shim to harvest commands.
- Tom: We don't use a shim, we monitor processes being launched.
- David: Why would that be different?
- Tom: If the build system constructs them, not an issue. If they are distributed in advance, then we don't have that option.
- David: For the IDE situation, I assume it is in a similar situation.
- Olga: yes. But similar issues happen when mixing cl and clang-cl.
- David: Two ways I've heard how to deal with this. One is with the gcc oracle system. So if you have two different compilers, both would call into the build system, and the build system could keep distinct caches for each compiler.
- Bryce: Next meeting in 2 weeks, tentatively Spencer and Bruno talking about BMI configuration.

§ 3.8. 2019-06-07 Meeting Minutes

ISO C++ SG15 Tooling Pre-Cologne Modules Tooling Interactions Telecon 2019-06-07

Chairing: Bryce Adelstein Lelbach

Minute Taker: Ben Craig

Attendance:

- Bryce Adelstein Lelbach.
- Ben Boeckel.
- Ben Craig.
- Corentin Jabot.

- David Blaikie.
- Gabriel Dos Reis [Gaby] (Microsoft).
- JF Bastien.
- Lukasz Menakiewicz (Microsoft).
- Mathias Stearn.
- Michael Spencer.
- Nathan Sidwell.
- Olga Arhipova (Microsoft).
- Rene Rivera.
- Richard Smith.
- Stephen Kelly.
- Steve Downey.
- Tom Honermann.

Chair Notes (Bryce Adelstein Leibach):

- Pre-Cologne Papers (Send Drafts to sg15@lists.isocpp.org by 2019-06-14):
- Ecosystem Technical Report Outline (Bryce).
- Summary and Minutes of Pre-Cologne SG15 Discussions (Bryce, Ben C).
- Dependency Metadata (Ben B).
- Compiled Module Reuse (Olga, Lukasz).
- Module Naming (Corentin Jabot).
- Modules Hello World (Gaby).
- Modules and Packaging (Richard).
- RFE: (Paper or info on) Implicit Modules (Michael).
- Michael gave a talk at LLVM Euro about this.

Minutes (Ben Craig):

- Bryce: Distribution of compiled modules. How are you dealing with different configurations?
- Michael: Not looking at distributing compiled modules. No plan on giving clang a stable module format. No compatibility between versions which would be confusing for devs. You need the

headers or module interface files to do anything. You have to be very careful about what module settings you are using when you are building, and it's much easier if you have the original file.

— Bryce: What about warnings?

— Michael: You should always get the same warnings. Only your own warning flags should affect things, regardless of how the module was built. We plan on serializing them into the AST. We'll have buckets for which warnings we will keep because some of them are expensive to create. The first time you import a module, we want to emit all the warnings that are applicable. For performance reasons, we don't want to keep everything from -Weverything.

— Tom: What about -wno-some-warning?

— Michael: We would filter the relevant warning out.

— Michael: Non header units may have a different model where you aren't viewing it as a header. It may be ok to say in that case that the module is its own entity. This would be steering away from implicit modules.

— Olga: What about clang-cl being able to read modules from MSVC?

— Richard: Warning mapping explicit module builds. Explicit module builds, when we build a module, the warnings used to build that module are used, not the warnings when consumed. So a template instantiated by a consumer would get the producing modules warnings. Explicit and implicit models are different.

— Michael: Agreed.

— Richard: Maybe that isn't the criterion, but that's the way we are doing things now.

— Bryce: Would be nice if it were easy to explicitly control what model they got.

— Gaby: clang-cl and MSVC compat is a discussion going on internally. We've been talking with Richard.

— Gaby: Clang has experience with implicit module builds, but msvc doesn't. The MSVC implementation of the TS assumes explicit builds. May be ok for usage at scale, but isn't necessarily good for small examples. Is there any way that clang people could write an experience paper?

— Michael: Gave a talk at LLVM Euro. I don't know what you are looking for in terms of how that model looks.

— Gaby: Interaction with build systems. The implicit build may conflict with the build system, especially around caching.

— Michael: Should generally be hidden. From build systems.

— Gaby: Scalability?

- Michael: There are issues, I think a CppCon talk about it.
- Bryce: Some implementations want distributing, some do not want people to do that. Do we think it's a problem that there is divergence. Is that fine?
- Gaby: There's a notion of reusing which could just be for a given compiler release. Can I reuse those. That's separate from distribution of artifacts that will outlive the current compiler series.
- Bryce: If I'm a user of MSVC using C++20, is your advice that I shouldn't ship BMIs for my common configurations, or is that fine?
- Gaby: Current advice is that for a given compiler release it's ok, so we can avoid rebuilding everything repeatedly. But you should also ship the source code. Even when the format of the BMI is published, we will still recommend that source code be distributed.
- Bryce: For clang, must have the source code for the interface unit.
- JF: Three ways to distribute modules. Pure source. Make internal format stable. Or intermediate where some is stable and some is not. For example, description of struct is described in a stable way. With C++, that ends up being an awful lot of stuff though. So technically possible to have a binary module attached with pared down version of the source code, but that is pretty big. There's a talk by swift people about how they are doing it (<https://developer.apple.com/videos/play/wwdc2019/416/>). We don't want to serialize the AST. It's possible, but it's big and may have more than you want.
- Gaby: I don't know if the size is that big or not. I'm not deterred by the size at this point. I'm more interested in the ability of developers to not need to recompile the same thing repeatedly.
- JF: What I'm talking about allows for that. The expensive part isn't source -> ast, but ast -> compiled stuff. You save a bit of time using AST, but you don't save the expensive part. Would like to see tools that pare down what is exported. The big things may not just be lots of bytes, but more things than you would expect, and you'll see unexpected things.
- Mathias: How does the size compare to headers? Will it be larger or roughly the same.
- JF: It's easier to use a template that uses a name inside of it. When a user gives some other name, it pulls in a whole lot of other stuff. Hard for a compiler to prove that similarly named things won't be pulled in. Not sure how things like reflection gets exposed through tree shaking or whatnot.
- Michael: At the worst, it's everything that's public in your module interface unit.
- Corentin: I like to think of it as compiling as-if from by source, but I think of that as caching, possibly even at the company level. Using that model to distribute modules is probably fine. Making things stable / never able to change is a problem though, and I'm opposed to that.
- Bryce: The implementers are all saying you must distribute some form of source, and that you can provide interfaces as an optimization.

- Gaby: Your summary is fantastic Corentin. This is why we need to split the discussion into distribution and reuse. Guaranteeing compatibility has a big cost, but we still want to be able to enable reuse.
- Bryce: What is the clang / llvm plan for distributing default module maps. C++ implementation may not be well suited to do, but the OS is in a good position to do. What are vendors going to need to do to modularize themselves.
- Richard: Assume something like clang's module map system. You need some kind of file accompanying a package that says which headers are to be treated as header units. Any side information you need to build in that mode will be there. GLIBC in particular has information that varies slightly between version, as info moves into and out of "bits" headers. We don't think the compiler is in a good position to distribute module description files for glibc, because we would need different ones for different versions, and need to know which one you have on a given machine. glibc would need to provide that data. We may be able to provide examples so that distributions could provide that. Hoping there will be sufficient adoption and cross vendor description compat so that packages can provide that information themselves.
- Mathias: Has there been any discussion with wg14 about C modules so that we have a good compat story with their library.
- JF: No. No discussion in wg14 in the last year. You could join the interop list and ask.
- Bryce: Hard to execute, since we would basically just tell them here's the design, please adopt it, without being able to take feedback.
- JF: Question is whether modules is worth their time. My concern is that no one will attend, or that they will make something slightly incompatible. If we don't participate, incompatibilities are very likely.
- Gaby: I'm nervous about C taking things from C++ without modifying them. All the prior cases resulted in modifications except for // comments.
- Stephen K: Packaging and libc, distributors would need to provide module map files. All the maps needed the same name... how would that work. Clang can look for them implicitly, but clang allows you to name module maps on the command line. Not sure that module maps are where we want to go, that's something that SG15 should discuss.
- JF: Apple ships implicit modules maps for some of the frameworks we ship. It's not perfect, but it does some good work. Some of the details end up leaking. It's hard to break up some of the dependencies. We also have a giant codebase with all the frameworks we expose. Exposing contents as modules can serve as an example of how to migrate existing code. We (Apple) own the versions as the platform vendor, but the clang project isn't the right place to do this.
- Stephen K: https://bugs.llvm.org/show_bug.cgi?id=21593.

- Corentin: Modules need to be authored by the people maintaining the code, and not by someone else. Maintainer of a package should provide the modules. Not a good idea for someone that doesn't maintain a library to provide a module for it. People that aren't the stdlib shouldn't implement modules for the stdlib.
- Ben B: Not so bad if I modularize (for example) libpng, but it is bad if I choose to distribute it.
- JF: Bryce: The Apple paper was in the post-Kona mailing, P1482R0.